

Звіт
до Лабораторної роботи №3
Об'єктно-орієнтоване програмування. Java
Модульне тестування в JUnit Jupiter

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Сформувати практичні навички модульного тестування Java-коду засобами JUnit Jupiter, організації тестового Maven-проєкту, побудови фікстур, перевірки позитивних і негативних сценаріїв, а також зіставлення Maven-підходу з Gradle як альтернативним інструментом збирання.

2. Завдання роботи

1. Створити окремий Maven-проєкт для модульного тестування предметної моделі «Спортивні секції».
2. Підготувати модульні тести для всіх сутностей системи (SportsClub, SportSection, Student, Enrollment, Coach, Schedule).
3. Використати фікстури через анотацію @BeforeEach для забезпечення незалежного стартового стану.
4. Перевірити коректність роботи конструкторів, обробку винятків (негативні сценарії) та бізнес-логіку системи.

3. Роль модульних тестів для моделі «Спортивні секції»

Для розробленої моделі модульні тести виконують критичну функцію перевірки інваріантів та бізнес-правил. Вони гарантують, що:

- Неможливо створити сутності (студента, тренера, секцію) з некоректними або порожніми даними (наприклад, без ПІБ або назви).
- Головний клас-контейнер `SportsClub` правильно агрегує дані та керує записами.
- Виконуються предметні обмеження: наприклад, система блокує спробу записати студента на секцію, реєстрацію до якої вже закрито, та генерує відповідний виняток `IllegalStateException`.
Без тестів ці правила існували б лише в теорії, а тести дозволяють автоматично перевіряти їх при кожній зміні коду.

4. Опис реалізованих тестових сценаріїв

Тестування було розділено на ізольовані класи згідно з принципом єдиної відповідальності:

- **Позитивні сценарії:** Перевірено успішне створення всіх об'єктів моделі (`Student`, `Coach`, `SportSection` тощо) та коректність роботи методів доступу (геттерів). Перевірено успішне оформлення запису (`Enrollment`) відкритої секції у класі `SportsClub`.
- **Негативні сценарії:** За допомогою методу `assertThrows(...)` перевірено реакцію системи на некоректні дії. Наприклад, створення `SportSection` із порожньою назвою викидає `IllegalArgumentException`, а спроба виклику `enrollStudent` для закритої секції перехоплюється як `IllegalStateException`.
- **Використання фікстур:** У класі `SportsClubTest` застосовано анотацію `@BeforeEach`. Вона дозволяє перед запуском кожного окремого тесту наново створювати чистий об'єкт клубу, додавати туди секцію та студента. Це забезпечує незалежність тестів один від одного (зміна стану в одному тесті не ламає інший).

5. Gradle як альтернативний інструмент збирання

У даній роботі для конфігурації проєкту (завантаження бібліотек JUnit) використовувався **Maven** (файл `pom.xml`). Альтернативою йому є система збирання **Gradle**. Основні відмінності:

- **Формат запису:** Maven використовує декларативний формат XML, який є більш строгим та багатослівним. Gradle використовує Groovy або Kotlin DSL, що робить конфігурацію коротшою та гнучкішою.

- **Швидкодія:** Gradle є значно швидшим за рахунок інкрементальних збірок (він не перезбирає те, що не змінилося з минулого разу) та використання фонового процесу (Gradle Daemon).
 - **Гнучкість:** У Maven життєвий цикл (Lifecycle) жорстко зафіксований, тоді як Gradle дозволяє писати власні скрипти для складних процесів збірки.
- Незважаючи на переваги Gradle у швидкодії, Maven залишається стандартом де-факто для багатьох корпоративних Java-проектів завдяки своїй простоті та стандартизації.

6. Демонстрація роботи

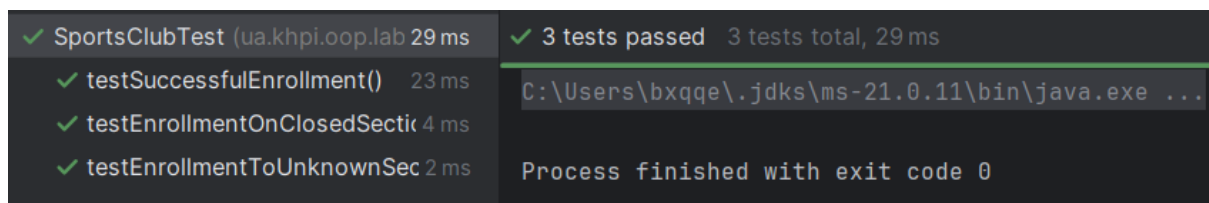


Рисунок 1

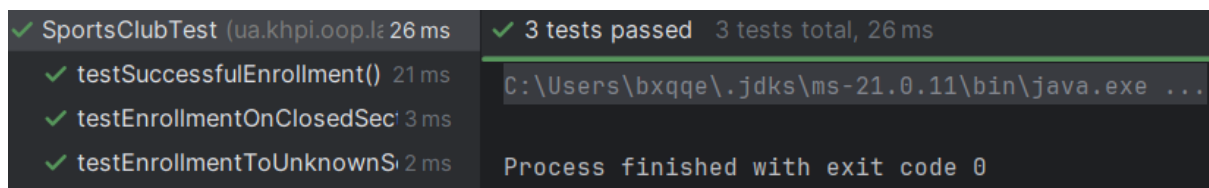


Рисунок 2

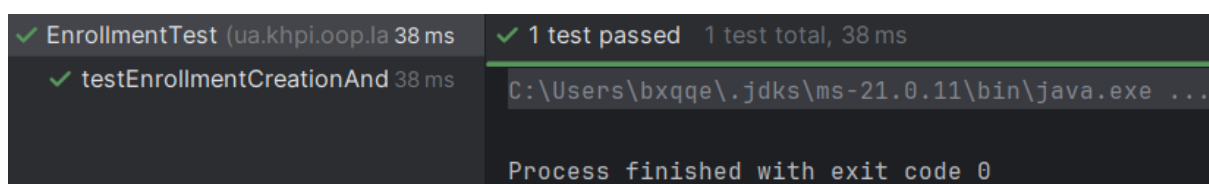


Рисунок 3

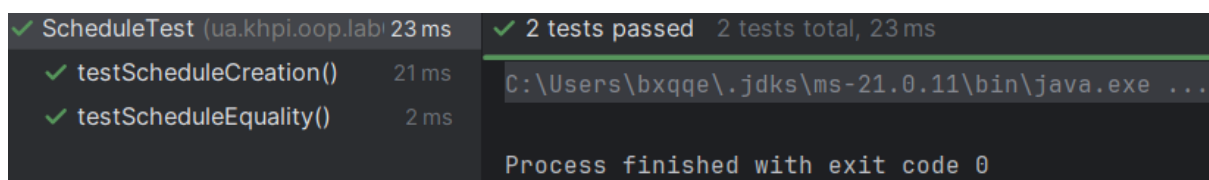


Рисунок 4

```
✓ SportSectionTest (ua.khpi.oop 23 ms) ✓ 3 tests passed 3 tests total, 23 ms
  ✓ testSectionCreationAndGet 21 ms
  ✓ testChangeRegistrationStatus 1 ms
  ✓ testExceptionOnEmptyName 1 ms
C:\Users\bxqqe\.jdk\ms-21.0.11\bin\java.exe ...
Process finished with exit code 0
```

Рисунок 5

```
✓ StudentTest (ua.khpi.oop.lab0: 21 ms) ✓ 2 tests passed 2 tests total, 21 ms
  ✓ testStudentCreation() 18 ms
  ✓ testEmptyTicketIdThrowsException 3 ms
C:\Users\bxqqe\.jdk\ms-21.0.11\bin\java.exe ...
Process finished with exit code 0
```

Рисунок 6

7. Висновок

У ході виконання лабораторної роботи було створено Maven-проект та налаштовано середовище для модульного тестування за допомогою JUnit Jupiter. Предметну модель «Спортивні секції» було повністю покрито тестами. На практиці закріплено використання фікстур (анотація `@BeforeEach`) для підготовки тестових даних. Успішно реалізовано перевірки як позитивних, так і негативних сценаріїв, що підтверджує стійкість розробленої бізнес-логіки до некоректних вхідних даних та порушень правил предметної області.